

05_AI_DEVELOPMENT.md

SpeechStudio AI Development Rules

Version: 1.0

Status: Approved

1. Purpose

This document defines the mandatory development rules for every AI coding assistant contributing to SpeechStudio.

The goal is to ensure a consistent architecture, predictable codebase and maintainable implementation.

The AI shall follow these rules before creating or modifying source code.

2. Specification First

The specification is the primary source of truth.

Implementation shall follow the specification.

If implementation and specification differ

the specification wins.

The AI shall never invent functionality that is not described in the specification.

3. Supported Technologies

Python

PyTorch

Transformers

PySide6

SoundFile

NumPy

The AI shall not introduce additional frameworks without explicit approval.

4. Architecture Rules

The UI shall never communicate directly with Transformers.

The UI communicates only with the Engine.

The Engine communicates only with the Backend.

The Backend communicates with Higgs Audio.

Every layer has one responsibility.

5. Single Responsibility

Each class shall have one clear responsibility.

Examples

ModelManager

loads the model only

PromptBuilder

builds prompts only

VoiceManager

manages voices only

AudioManager

handles audio only

HistoryManager

stores history only

6. Project Structure

All source code remains inside the project directory.

Relative paths only.

Absolute paths are prohibited.

Never reference

C:

D:

F:

or user specific folders.

7. Portable Design

SpeechStudio is a portable application.

The AI shall never write outside the project directory.

Forbidden locations include

AppData

Documents

Desktop

Registry

Temporary Windows folders

User profile directories

8. Model Lifecycle

Exactly one model instance exists.

Exactly one tokenizer exists.

The model is loaded once.

The tokenizer is loaded once.

The model remains resident until application shutdown.

9. Prompt Construction

Only PromptBuilder may create Higgs prompt tokens.

The UI must never concatenate token strings.

The Engine must never manually edit prompt syntax.

10. Voice Handling

VoiceManager owns

Reference audio

Reference transcript

Voice metadata

No other module shall modify voice files directly.

11. Threading

Long running operations shall never execute on the UI thread.

Speech generation executes on a worker thread.

Audio playback executes independently.

The interface must remain responsive.

12. Error Handling

The AI shall never suppress exceptions.

Every exception shall be

logged

reported

handled gracefully

The application must not terminate unexpectedly.

13. Logging

Every major subsystem shall write structured log messages.

Logs shall include

Timestamp

Component

Severity

Message

Optional stack trace

14. File Formats

Configuration

JSON

Generated Audio

WAV

Reference Audio

WAV

Logs

TXT

The AI shall not introduce proprietary file formats.

15. Code Style

Readable code.

Small functions.

Clear class names.

Explicit variable names.

No duplicated logic.

No unnecessary abstraction.

No premature optimization.

16. Dependencies

The AI shall reuse existing libraries whenever possible.

Do not replace an existing dependency without approval.

Do not introduce cloud services.

Do not introduce telemetry.

Do not introduce analytics.

17. UI Rules

The UI shall contain no business logic.

The UI forwards requests to the Engine.

The UI displays results.

The UI never performs speech generation.

18. Engine Rules

The Engine owns

Generation

Validation

History updates

Output writing

Prompt execution

The Engine never manipulates widgets.

19. Settings Rules

Every user configurable option shall be editable through the graphical interface.

Users shall never edit Python files to change application behaviour.

Settings are stored as JSON.

20. Performance Rules

Reuse loaded resources.

Avoid repeated allocations.

Avoid repeated model loading.

Release temporary memory after generation.

Do not block the interface.

21. Documentation Rules

Whenever functionality changes

The corresponding specification document shall be updated before implementation.

The AI shall not create undocumented functionality.

22. User Experience Rules

The user should never need to

Open a terminal

Activate a virtual environment

Edit Python files

Edit JSON manually

Edit prompt tokens manually

SpeechStudio shall expose these features through the graphical interface.

23. Scope Control

The AI shall implement only functionality defined in the specification.

The AI shall not add

new speech engines

cloud features

plugin systems

web interfaces

REST APIs

or unrelated functionality

unless explicitly requested.

24. Completion Criteria

A task is complete only when

The implementation follows the specification.

The code is readable.

The application remains portable.

The existing architecture is preserved.

No undocumented behaviour has been introduced.